

routeOptimizer.ts (Rewritten — Mail Carrier)

```

/**
 * Advanced Route Optimization Engine
 *
 * Implements a mail-carrier (boustrophedon) walking pattern:
 * 1. Group properties by normalized street name.
 * 2. Order streets via nearest-neighbor TSP on street centroids + 2-Opt.
 * 3. Within each street, walk one side low! high then the other side high! low
 * (or reversed, depending on which end of the street you arrive from).
 * 4. Apply intra-street 2-Opt to tighten each side independently.
 *
 * This eliminates the cross-street backtracking visible as yellow lines
 * crossing back over themselves on the map.
 */
import { filterByStreetCooldown, COOLDOWN_CONFIG } from './territoryLogic';
import { latLngToCell, gridDisk } from 'h3-js';
import { batchScoreProperties, ownershipDurationScore, SCORING_CONSTANTS } from './leadScoring';

// -----
// Types
// -----

interface Property {
  lat: number;
  lng: number;
  street_name?: string;
  house_number?: number | string;
  address_hash?: string;
  id?: string;
  legacy_hash?: string;
  effective_status?: string;
  sold_date?: string;
  price?: number;
  property_type?: string;
  zip_code?: string | number;
  year_built?: number;
  lot_size?: number;
  propensity?: number;
  score?: number;
  cluster?: number;
  is_ghost?: boolean;
  full_address?: string;
  [key: string]: unknown;
}

interface StartLocation {
  lat: number;
  lng: number;
}

```

```
}
```

```
interface RouteResult {  
  id: string;  
  name: string;  
  properties: Property[];  
  houseCount: number;  
  streetCount: number;  
  streets: string[];  
  totalDistance: number;  
  distanceFromStart: number;  
  totalScore: number;  
  avgScore: number;  
  competitivenessScore: number;  
  status: string;  
  completedCount: number;  
}
```

```
interface LearnedWeights {  
  age_gt_10_weight?: number;  
  price_gt_300k_weight?: number;  
  single_family_weight?: number;  
  recent_sale_weight?: number;  
  high_value_weight?: number;  
  large_lot_weight?: number;  
}
```

```
// -----  
// Distance utilities  
// -----
```

```
/** Haversine distance in miles between two lat/lng points. */  
function calculateDistance(lat1: number, lng1: number, lat2: number, lng2: number): number {  
  const R = 3959;  
  const dLat = (lat2 - lat1) * Math.PI / 180;  
  const dLng = (lng2 - lng1) * Math.PI / 180;  
  const a =  
    Math.sin(dLat / 2) * Math.sin(dLat / 2) +  
    Math.cos(lat1 * Math.PI / 180) * Math.cos(lat2 * Math.PI / 180) *  
    Math.sin(dLng / 2) * Math.sin(dLng / 2);  
  const c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));  
  return R * c;  
}
```

```
/** Fast squared planar distance (no sqrt) — used for comparisons only. */  
function calculateDistanceSquaredFast(lat1: number, lng1: number, lat2: number, lng2: number): number {  
  const x = (lng2 - lng1) * Math.cos((lat1 + lat2) / 2 * Math.PI / 180);
```

```
const y = lat2 - lat1;
return x * x + y * y;
}
```

```
/** Fast planar distance in miles — suitable for short distances (<50 mi). */
function calculateDistanceFast(lat1: number, lng1: number, lat2: number, lng2: number): number {
  const x = (lng2 - lng1) * Math.cos((lat1 + lat2) / 2 * Math.PI / 180);
  const y = lat2 - lat1;
  return Math.sqrt(x * x + y * y) * 69;
}
```

```
/** Bearing in degrees (0–360) from point 1 to point 2. */
function calculateBearing(lat1: number, lng1: number, lat2: number, lng2: number): number {
  const lat1r = lat1 * Math.PI / 180;
  const lat2r = lat2 * Math.PI / 180;
  const dLng = (lng2 - lng1) * Math.PI / 180;
  const y = Math.sin(dLng) * Math.cos(lat2r);
  const x = Math.cos(lat1r) * Math.sin(lat2r) - Math.sin(lat1r) * Math.cos(lat2r) * Math.cos(dLng);
  return (Math.atan2(y, x) * 180 / Math.PI + 360) % 360;
}
```

```
// -----
// Step 1 — normalizeStreetName
// -----
```

```
const DIRECTIONAL_PREFIXES = new Set(['N', 'S', 'E', 'W', 'NE', 'NW', 'SE', 'SW']);
```

```
const STREET_SUFFIXES = new Set([
  'ST', 'AVE', 'BLVD', 'DR', 'RD', 'LN', 'WAY', 'CT', 'PL', 'CIR',
  'STREET', 'AVENUE', 'BOULEVARD', 'DRIVE', 'ROAD', 'LANE', 'COURT', 'PLACE', 'CIRCLE',
]);
```

```
/**
 * Normalize a street name for grouping purposes.
 * - Uppercases and trims.
 * - Strips leading directional prefixes (N, S, E, W, NE, NW, SE, SW).
 * - Strips trailing type suffixes (ST, AVE, BLVD, etc.).
 *
 * Examples:
 *   "N Oak St"    !> "OAK"
 *   "Oak Street"  !> "OAK"
 *   "SW 3rd Ave"  !> "3RD"
 */
```

```
export function normalizeStreetName(name: string): string {
  if (!name || !name.trim()) return '__UNKNOWN__';
  const tokens = name.toUpperCase().trim().split(/ /);
  // Strip leading directional prefix
```

```

    if (tokens.length > 1 && DIRECTIONAL_PREFIXES.has(tokens[0])) {
        tokens.shift();
    }
    // Strip trailing suffix
    if (tokens.length > 1 && STREET_SUFFIXES.has(tokens[tokens.length - 1])) {
        tokens.pop();
    }
    return tokens.join(' ') || '___UNKNOWN___';
}

// -----
// Step 2 — groupByStreet
// -----

/**
 * Group properties by their normalized street name.
 * Properties with a missing or empty street_name are placed in "___UNKNOWN___".
 */
function groupByStreet(properties: Property[]): Map<string, Property[]> {
    const groups = new Map<string, Property[]>();
    for (const p of properties) {
        const key = normalizeStreetName(p.street_name || "");
        if (!groups.has(key)) groups.set(key, []);
        groups.get(key)!.push(p);
    }
    return groups;
}

// -----
// Step 3 — orderStreetsByNearestNeighbor
// -----

interface StreetCentroid {
    key: string;
    lat: number;
    lng: number;
}

/** Compute the geographic centroid (mean lat/lng) of a set of properties. */
function computeCentroid(props: Property[]): { lat: number; lng: number } {
    let sumLat = 0;
    let sumLng = 0;
    for (const p of props) {
        sumLat += p.lat;
        sumLng += p.lng;
    }
    return { lat: sumLat / props.length, lng: sumLng / props.length };
}

```

```
}
```

```
/**
```

```
 * Apply 2-Opt to an ordered list of street centroids.
```

```
 * Only reorders streets — never mixes properties across streets.
```

```
 * Safe to run on the centroid sequence because it only swaps street order.
```

```
*/
```

```
function apply2OptCentroids(centroids: StreetCentroid[]): StreetCentroid[] {
```

```
  if (centroids.length < 4) return centroids;
```

```
  const route = [...centroids];
```

```
  let improved = true;
```

```
  let iterations = 0;
```

```
  const maxIterations = 50;
```

```
  while (improved && iterations < maxIterations) {
```

```
    improved = false;
```

```
    iterations++;
```

```
    for (let i = 0; i < route.length - 2; i++) {
```

```
      for (let j = i + 2; j < route.length; j++) {
```

```
        const a = route[i];
```

```
        const b = route[i + 1];
```

```
        const c = route[j];
```

```
        const d = j + 1 < route.length ? route[j + 1] : null;
```

```
        const before =
```

```
          calculateDistanceFast(a.lat, a.lng, b.lat, b.lng) +
```

```
          (d ? calculateDistanceFast(c.lat, c.lng, d.lat, d.lng) : 0);
```

```
        const after =
```

```
          calculateDistanceFast(a.lat, a.lng, c.lat, c.lng) +
```

```
          (d ? calculateDistanceFast(b.lat, b.lng, d.lat, d.lng) : 0);
```

```
        if (after < before - 1e-9) {
```

```
          // Reverse the segment between i+1 and j (inclusive)
```

```
          const segment = route.slice(i + 1, j + 1).reverse();
```

```
          route.splice(i + 1, segment.length, ...segment);
```

```
          improved = true;
```

```
        }
```

```
      }
```

```
    }
```

```
  }
```

```
  return route;
```

```
}
```

```
/**
```

```
 * Order street groups using nearest-neighbor TSP on street centroids,
```

```
 * then tighten with 2-Opt on the centroid sequence.
```

```
 *
```

```
 * @param streetGroups Map of normalized street key!' properties
```

```
 * @param startLat Starting latitude (rep's current position)
```

```
 * @param startLng Starting longitude
```

```
* @returns Ordered array of street keys
```

```
*/
```

```
function orderStreetsByNearestNeighbor(
  streetGroups: Map<string, Property[]>,
  startLat: number,
  startLng: number,
): string[] {
  // Build centroid list (exclude __UNKNOWN__ — handled separately)
  const centroids: StreetCentroid[] = [];
  for (const [key, props] of streetGroups.entries()) {
    if (key === '__UNKNOWN__') continue;
    const c = computeCentroid(props);
    centroids.push({ key, lat: c.lat, lng: c.lng });
  }

  if (centroids.length === 0) return streetGroups.has('__UNKNOWN__') ? ['__UNKNOWN__'] : [];

  // Nearest-neighbor TSP starting from startLat/startLng
  const unvisited = [...centroids];
  const ordered: StreetCentroid[] = [];
  let curLat = startLat;
  let curLng = startLng;

  while (unvisited.length > 0) {
    let bestIdx = 0;
    let bestDist = Infinity;
    for (let i = 0; i < unvisited.length; i++) {
      const d = calculateDistanceFast(curLat, curLng, unvisited[i].lat, unvisited[i].lng);
      if (d < bestDist) {
        bestDist = d;
        bestIdx = i;
      }
    }
    const next = unvisited.splice(bestIdx, 1)[0];
    ordered.push(next);
    curLat = next.lat;
    curLng = next.lng;
  }

  // Tighten with 2-Opt on centroid sequence
  const optimized = apply2OptCentroids(ordered);
  const result = optimized.map(c => c.key);

  // Append __UNKNOWN__ last
  if (streetGroups.has('__UNKNOWN__')) result.push('__UNKNOWN__');
  return result;
}
```

```

// -----
// Step 4 — boustrophedonStreet
// -----

/**
 * Order properties on a single street using the boustrophedon (mail-carrier) pattern:
 * - Walk one side of the street (odd or even house numbers) in one direction.
 * - Cross and walk the other side in the reverse direction.
 *   - Direction (low! high vs high! low) is determined by which end of the street
 *     is closest to the arrival point.
 *
 * @param properties All properties on this street
 * @param arrivalLat Latitude of the point we're arriving from
 * @param arrivalLng Longitude of the point we're arriving from
 * @returns Properties in boustrophedon walking order
 */
function boustrophedonStreet(
  properties: Property[],
  arrivalLat: number,
  arrivalLng: number,
): Property[] {
  if (properties.length === 0) return [];
  if (properties.length === 1) return [...properties];

  // Determine primary axis: sort by lat to find min/max coord house
  const byLat = [...properties].sort((a, b) => a.lat - b.lat);
  const minProp = byLat[0];
  const maxProp = byLat[byLat.length - 1];
  const bearing = calculateBearing(minProp.lat, minProp.lng, maxProp.lat, maxProp.lng);

  // E-W street: bearing 45-135 or 225-315 !' sort by longitude
  // N-S street: otherwise !' sort by latitude
  const isEW = (bearing >= 45 && bearing <= 135) || (bearing >= 225 && bearing <= 315);
  const axisKey: keyof Property = isEW ? 'lng' : 'lat';

  // Split into odd and even sides
  const oddSide: Property[] = [];
  const evenSide: Property[] = [];
  for (const p of properties) {
    const num = typeof p.house_number === 'string'
      ? parseInt(p.house_number, 10)
      : (p.house_number ?? 0);
    const n = isNaN(num as number) ? 0 : (num as number);
    if (n % 2 === 1) {
      oddSide.push(p);
    } else {

```



```

    evenSide.push(p);
  }
}

// Sort each side by axis coordinate (low to high)
const sortAsc = (arr: Property[]) =>
  arr.sort((a, b) => (a[axisKey] as number) - (b[axisKey] as number));
sortAsc(oddSide);
sortAsc(evenSide);

// Determine which end of the street is closest to arrival point
// Use the first property of the low-sorted combined list vs the last
const allSorted = [...properties].sort((a, b) => (a[axisKey] as number) - (b[axisKey] as number));
const lowEnd = allSorted[0];
const highEnd = allSorted[allSorted.length - 1];
const distToLow = calculateDistanceFast(arrivalLat, arrivalLng, lowEnd.lat, lowEnd.lng);
const distToHigh = calculateDistanceFast(arrivalLat, arrivalLng, highEnd.lat, highEnd.lng);
const arriveFromLow = distToLow <= distToHigh;

// Handle single-side streets
if (oddSide.length === 0) {
  return arriveFromLow ? evenSide : [...evenSide].reverse();
}
if (evenSide.length === 0) {
  return arriveFromLow ? oddSide : [...oddSide].reverse();
}

// Boustrophedon: walk first side in arrival direction, second side in reverse
if (arriveFromLow) {
  // Walk odd low! high, then even high! low
  return [...oddSide, ...[...evenSide].reverse()];
} else {
  // Walk odd high! low, then even low! high
  return [...[...oddSide].reverse(), ...evenSide];
}
}

// -----
// Step 5 — applyIntraStreet2Opt
// -----

/**
 * Apply 2-Opt optimization strictly within a single street's property list.
 * Because this only receives one street's properties, no cross-street swaps
 * are possible — street groupings are always preserved.
 *
 * Skips if fewer than 4 properties or more than 300 (safety limit).

```

```

*/
function applyIntraStreet2Opt(streetProperties: Property[]): Property[] {
  if (streetProperties.length < 4 || streetProperties.length > 300) {
    return streetProperties;
  }
  const route = [...streetProperties];
  let improved = true;
  let iterations = 0;
  const maxIterations = 50;

  while (improved && iterations < maxIterations) {
    improved = false;
    iterations++;
    for (let i = 1; i < route.length - 2; i++) {
      for (let j = i + 2; j < route.length; j++) {
        const p1 = route[i - 1];
        const p2 = route[j];
        const p3 = route[j];
        const p4 = j + 1 < route.length ? route[j + 1] : null;
        const before =
          calculateDistanceFast(p1.lat, p1.lng, p2.lat, p2.lng) +
          (p4 ? calculateDistanceFast(p3.lat, p3.lng, p4.lat, p4.lng) : 0);
        const after =
          calculateDistanceFast(p1.lat, p1.lng, p3.lat, p3.lng) +
          (p4 ? calculateDistanceFast(p2.lat, p2.lng, p4.lat, p4.lng) : 0);
        if (after < before - 1e-9) {
          const segment = route.slice(i, j + 1).reverse();
          route.splice(i, segment.length, ...segment);
          improved = true;
        }
      }
    }
  }
  return route;
}

// -----
// Step 6 — mailCarrierOrder (exported)
// -----

/**
 * Order properties using the mail-carrier (boustrophedon) algorithm.
 *
 * Algorithm:
 * 1. Group properties by normalized street name.
 * 2. Order streets via nearest-neighbor TSP on centroids + 2-Opt.
 * 3. Within each street, apply boustrophedon ordering based on arrival direction.

```

- * 4. Apply intra-street 2-Opt to tighten each street's sub-route.
- * 5. Concatenate all streets; append __UNKNOWN__ group at the end.

*

* @param properties Array of properties to order

* @param startLocation Optional starting position (rep's GPS location)

* @returns Flat array of properties in mail-carrier walking order

*/

```
export function mailCarrierOrder(
  properties: Property[],
  startLocation?: StartLocation | null,
): Property[] {
  if (!properties || properties.length === 0) return [];
  if (properties.length === 1) return [...properties];

  // Step 1 & 2: Group and order streets
  const streetGroups = groupByStreet(properties);

  // Determine start point
  let startLat: number;
  let startLng: number;
  if (startLocation && startLocation.lat && startLocation.lng) {
    startLat = startLocation.lat;
    startLng = startLocation.lng;
  } else {
    // Default: centroid of all properties
    const c = computeCentroid(properties);
    startLat = c.lat;
    startLng = c.lng;
  }

  const orderedStreetKeys = orderStreetsByNearestNeighbor(streetGroups, startLat, startLng);

  // Step 3, 4, 5: Process each street
  const result: Property[] = [];
  let arrivalLat = startLat;
  let arrivalLng = startLng;

  for (const key of orderedStreetKeys) {
    const streetProps = streetGroups.get(key);
    if (!streetProps || streetProps.length === 0) continue;

    // Boustrophedon ordering based on arrival point
    let ordered = boustrophedonStreet(streetProps, arrivalLat, arrivalLng);

    // Intra-street 2-Opt tightening
    ordered = applyIntraStreet2Opt(ordered);
```

```
result.push(...ordered);
```

```
// Update arrival point to the last property on this street
const last = ordered[ordered.length - 1];
arrivalLat = last.lat;
arrivalLng = last.lng;
}

return result;
}

// -----
// Scoring
// -----

/**
 * Score a single property for route prioritization.
 * Higher score = higher priority to visit.
 */
export function scoreProperty(
  property: Property,
  logs: unknown[] = [],
  neighborhoodStats: Record<string, number> = {},
  learnedWeights: LearnedWeights | null = null,
): number {
  let score = 100;
  if (property.effective_status === 'ELIGIBLE') score += 60;
  if (property.effective_status === 'UNVERIFIED') score += 40;
  if (property.effective_status === 'CALLBACK') score += 100;
  if (property.effective_status === 'NO_ANSWER') score += 30;
  if (property.effective_status === 'QUALIFIED') score += 80;
  if (property.effective_status === 'HARD_NO') return 0;
  if (property.effective_status === 'SOLD') {
    if (property.sold_date) {
      const monthsAgo = (Date.now() - new Date(property.sold_date).getTime()) / (1000 * 60 * 60 * 24 * 30);
      if (monthsAgo <= 3) score += 80;
      else if (monthsAgo <= 6) score += 60;
      else if (monthsAgo <= 12) score += 40;
      else score += 20;
    } else {
      score += 20;
    }
  }
  if (property.sold_date && property.price) {
    const soldDate = new Date(property.sold_date);
    const now = new Date();
    const yearsOwned = (now.getTime() - soldDate.getTime()) / (1000 * 60 * 60 * 24 * 365);
```

```

const appreciationFactor = 1 + (0.03 * yearsOwned);
const estValue = property.price * appreciationFactor;
const loanAmortization = Math.min(1, yearsOwned / 30);
const estEquity = estValue * (0.2 + (0.8 * loanAmortization));
if (estEquity > 200000) score += 60;
else if (estEquity > 100000) score += 30;
if (yearsOwned < 1) score -= 30;
else if (yearsOwned > 7) score += 40;
}
if (property.property_type) {
  const type = property.property_type.toLowerCase();
  if (type.includes('single')) score += 20;
  else if (type.includes('condo') || type.includes('town')) score -= 10;
  else if (type.includes('multi')) score += 10;
}
if (neighborhoodStats && property.lat && property.lng) {
  try {
    const h3Index = latLngToCell(property.lat, property.lng, 9);
    const zipHeat = neighborhoodStats[h3Index] || 0;
    score += Math.min(zipHeat * 5, 50);
  } catch (e) { /* ignore h3 errors */ }
}
if (logs && logs.length > 0) {
  const propHash = property.address_hash || property.id;
  const legacyHash = property.legacy_hash as string | undefined;
  const myLogs = (logs as Array<{ address_hash?: string; parsed_status?: string }>).filter(
    l => l.address_hash === propHash || (legacyHash && l.address_hash === legacyHash),
  );
  if (myLogs.length > 3) {
    score -= 60;
  } else if (myLogs.length === 1 && myLogs[0].parsed_status === 'NO_ANSWER') {
    score += 25;
  } else if (myLogs.length === 2 && myLogs.every(l => l.parsed_status === 'NO_ANSWER')) {
    score -= 10;
  }
}
}
if ((property.price ?? 0) > 1000000) score += 30;
if (learnedWeights) {
  if (property.year_built) {
    const age = new Date().getFullYear() - property.year_built;
    if (age > 10 && learnedWeights.age_gt_10_weight) score *= learnedWeights.age_gt_10_weight;
  }
  if ((property.price ?? 0) > 300000 && learnedWeights.price_gt_300k_weight) score *=
learnedWeights.price_gt_300k_weight;
  if (property.property_type?.toLowerCase().includes('single') && learnedWeights.single_family_weight) score *=
learnedWeights.single_family_weight;
  if (property.sold_date) {

```

```

    const yearsOwned = (Date.now() - new Date(property.sold_date).getTime()) / (1000 * 60 * 60 * 24 * 365);
    if (yearsOwned <= 3 && learnedWeights.recent_sale_weight) score *= learnedWeights.recent_sale_weight;
  }
  if ((property.price ?? 0) > 750000 && learnedWeights.high_value_weight) score *=
learnedWeights.high_value_weight;
  if ((property.lot_size ?? 0) > 10890 && learnedWeights.large_lot_weight) score *= learnedWeights.large_lot_weight;
}
return Math.max(0, Math.round(score));
}

// -----
// k-Means clustering (retained for future multi-route support)
// -----

function kMeansPlusPlusInit(items: Property[], numClusters: number): Array<{ lat: number; lng: number }> {
  const centroids: Array<{ lat: number; lng: number }> = [];
  const first = items[Math.floor(Math.random() * items.length)];
  centroids.push({ lat: first.lat, lng: first.lng });
  for (let c = 1; c < numClusters; c++) {
    const distances = items.map(p => {
      let minD = Infinity;
      centroids.forEach(cen => {
        const d = calculateDistanceSquaredFast(p.lat, p.lng, cen.lat, cen.lng);
        if (d < minD) minD = d;
      });
      return minD;
    });
    const totalD = distances.reduce((s, d) => s + d, 0);
    if (totalD === 0) {
      const pick = items[Math.floor(Math.random() * items.length)];
      centroids.push({ lat: pick.lat, lng: pick.lng });
      continue;
    }
    let r = Math.random() * totalD;
    for (let i = 0; i < distances.length; i++) {
      r -= distances[i];
      if (r <= 0) {
        centroids.push({ lat: items[i].lat, lng: items[i].lng });
        break;
      }
    }
  }
  if (centroids.length <= c) {
    centroids.push({ lat: items[items.length - 1].lat, lng: items[items.length - 1].lng });
  }
}
return centroids;
}

```

```

function kMeansClustering(properties: Property[], numClusters: number): Property[] {
  if (properties.length <= numClusters) {
    return properties.map((p, i) => ({ ...p, cluster: i }));
  }
  const items: Property[] = properties.map(p => ({ ...p }));
  let centroids = kMeansPlusPlusInit(items, numClusters);
  let iterations = 0;
  const maxIterations = 20;
  let changed = true;
  while (changed && iterations < maxIterations) {
    changed = false;
    iterations++;
    items.forEach(prop => {
      let minDist = Infinity;
      let bestCluster = 0;
      centroids.forEach((centroid, idx) => {
        const dist = calculateDistanceSquaredFast(prop.lat, prop.lng, centroid.lat, centroid.lng);
        if (dist < minDist) {
          minDist = dist;
          bestCluster = idx;
        }
      });
      if (prop.cluster !== bestCluster) {
        changed = true;
        prop.cluster = bestCluster;
      }
    });
    centroids = centroids.map((_, idx) => {
      const clusterProps = items.filter(p => p.cluster === idx);
      if (clusterProps.length === 0) return centroids[idx];
      let totalWeight = 0;
      let wLat = 0;
      let wLng = 0;
      clusterProps.forEach(p => {
        const w = Math.max(0.1, p.propensity ?? (p.score !== null ? p.score / 400 : 0.5));
        wLat += w * p.lat;
        wLng += w * p.lng;
        totalWeight += w;
      });
      return { lat: wLat / totalWeight, lng: wLng / totalWeight };
    });
  }
  return items;
}

// -----

```

```
// Fatigue-aware front-loading
```

```
// -----
```

```
const FATIGUE_FRONT_LOAD_STOPS = 22;
```

```
const FRONT_LOAD_PROPENSITY_PERCENTILE = 0.30;
```

```
const DEFAULT_MAX_DISTANCE_INCREASE = 0.12;
```

```
/**
```

```
 * Move the highest-propensity properties to the first ~22 stops of the route
```

```
 * so reps encounter the best leads while energy is highest.
```

```
 * Respects a 12% maximum total-distance increase constraint.
```

```
*/
```

```
function fatigueAwareFrontLoad(route: Property[]): Property[] {  
  if (route.length <= FATIGUE_FRONT_LOAD_STOPS) return route;  
  const routeDist = (r: Property[]) => {  
    let d = 0;  
    for (let i = 0; i < r.length - 1; i++) {  
      d += calculateDistanceFast(r[i].lat, r[i].lng, r[i + 1].lat, r[i + 1].lng);  
    }  
    return d;  
  };  
  const baselineDist = routeDist(route);  
  const scored = route.map((p, idx) => ({ idx, propensity: p.propensity ?? p.score ?? 0 }));  
  scored.sort((a, b) => b.propensity - a.propensity);  
  const topCount = Math.ceil(route.length * FRONT_LOAD_PROPENSITY_PERCENTILE);  
  const topIndices = new Set(scored.slice(0, topCount).map(s => s.idx));  
  const toMove: number[] = [];  
  for (let i = FATIGUE_FRONT_LOAD_STOPS; i < route.length; i++) {  
    if (topIndices.has(i)) toMove.push(i);  
  }  
  if (toMove.length === 0) return route;  
  const result = [...route];  
  let insertPos = 1;  
  for (const fromIdx of toMove) {  
    const currentIdx = result.indexOf(route[fromIdx]);  
    if (currentIdx < 0 || currentIdx <= insertPos) continue;  
    if (insertPos >= FATIGUE_FRONT_LOAD_STOPS) break;  
    const item = result.splice(currentIdx, 1)[0];  
    result.splice(insertPos, 0, item);  
    insertPos++;  
    const newDist = routeDist(result);  
    if ((newDist - baselineDist) / baselineDist > DEFAULT_MAX_DISTANCE_INCREASE) {  
      result.splice(insertPos - 1, 1);  
      result.splice(currentIdx, 0, item);  
      insertPos--;  
      break;  
    }  
  }  
}
```



```

    }
    return result;
}

// -----
// Legacy route ordering (used by optimizeRouteByDistance only)
// -----

function optimizeRouteOrder(
  properties: Property[],
  startLat: number | null = null,
  startLng: number | null = null,
  minimizeTurns = false,
): Property[] {
  if (properties.length === 0) return [];
  const unvisited = [...properties];
  const route: Property[] = [];
  let current: Property;
  let currentBearing: number | null = null;

  if (startLat !== null && startLng !== null) {
    let nearestIdx = 0;
    let minScore = Infinity;
    unvisited.forEach((prop, idx) => {
      const dist = calculateDistanceFast(startLat, startLng, prop.lat, prop.lng);
      if (dist < minScore) {
        minScore = dist;
        nearestIdx = idx;
      }
    });
    current = unvisited.splice(nearestIdx, 1)[0];
  } else {
    current = unvisited.shift()!;
  }
  route.push(current);

  while (unvisited.length > 0) {
    let bestIdx = 0;
    let bestScore = Infinity;
    unvisited.forEach((prop, idx) => {
      const dist = calculateDistanceFast(current.lat, current.lng, prop.lat, prop.lng);
      let score = dist;
      if (minimizeTurns && currentBearing !== null) {
        const newBearing = calculateBearing(current.lat, current.lng, prop.lat, prop.lng);
        const turnAngle = Math.abs(newBearing - currentBearing);
        const normalizedTurn = turnAngle > 180 ? 360 - turnAngle : turnAngle;
        if (normalizedTurn > 45) {

```

```

        score += (normalizedTurn / 180) * 0.5;
    }
}
if (score < bestScore) {
    bestScore = score;
    bestIdx = idx;
}
});
const nextProp = unvisited.splice(bestIdx, 1)[0];
currentBearing = calculateBearing(current.lat, current.lng, nextProp.lat, nextProp.lng);
current = nextProp;
route.push(current);
}
return route;
}

// -----
// Legacy 2-Opt and Link Swap (used by optimizeRouteByDistance only)
// -----

function apply2Opt(route: Property[]): Property[] {
    if (route.length < 4) return route;
    if (route.length > 300) return route;
    let improved = true;
    const maxIterations = 50;
    let iterations = 0;
    const dummy = { isDummy: true } as unknown as Property;
    const currentRoute = [...route, dummy];
    const dist = (pA: Property, pB: Property): number => {
        if ((pA as unknown as { isDummy?: boolean }).isDummy || (pB as unknown as { isDummy?: boolean }).isDummy)
            return 0;
        return calculateDistanceFast(pA.lat, pA.lng, pB.lat, pB.lng);
    };
    while (improved && iterations < maxIterations) {
        improved = false;
        iterations++;
        for (let i = 1; i < currentRoute.length - 2; i++) {
            for (let j = i + 2; j < currentRoute.length - 1; j++) {
                const p1 = currentRoute[i];
                const p2 = currentRoute[i + 1];
                const p3 = currentRoute[j];
                const p4 = currentRoute[j + 1];
                const currentDist = dist(p1, p2) + dist(p3, p4);
                const newDist = dist(p1, p3) + dist(p2, p4);
                if (newDist < currentDist) {
                    const segment = currentRoute.slice(i + 1, j + 1).reverse();
                    currentRoute.splice(i + 1, segment.length, ...segment);
                }
            }
        }
    }
}

```

```

    improved = true;
  }
}
}
const sIdx = currentRoute.findIndex(p => (p as unknown as { isDummy?: boolean }).isDummy);
currentRoute.splice(sIdx, 1);
route.length = 0;
route.push(...currentRoute);
return route;
}

```

```

function applyLinkSwap(route: Property[]): Property[] {
  if (route.length < 4) return route;
  if (route.length > 300) return route;
  let improved = true;
  let iterations = 0;
  const maxIterations = 30;
  const dist = (a: Property | null, b: Property | null): number => {
    if (!a || !b) return 0;
    return calculateDistanceFast(a.lat, a.lng, b.lat, b.lng);
  };
  while (improved && iterations < maxIterations) {
    improved = false;
    iterations++;
    for (let segLen = 1; segLen <= 2; segLen++) {
      for (let i = 1; i < route.length - segLen; i++) {
        const prev = route[i - 1];
        const segEnd = route[i + segLen - 1];
        const next = route[i + segLen] ?? null;
        const removeCost = dist(prev, route[i]) + (next ? dist(segEnd, next) : 0);
        const removeGain = next ? dist(prev, next) : 0;
        const removalSaving = removeCost - removeGain;
        for (let j = 0; j < route.length - 1; j++) {
          if (j >= i - 1 && j <= i + segLen - 1) continue;
          const insertCost = dist(route[j], route[i]) + dist(segEnd, route[j + 1]) - dist(route[j], route[j + 1]);
          if (removalSaving - insertCost > 0.001) {
            const segment = route.splice(i, segLen);
            const insertIdx = j < i ? j + 1 : j + 1 - segLen;
            route.splice(insertIdx, 0, ...segment);
            improved = true;
            break;
          }
        }
      }
      if (improved) break;
    }
    if (improved) break;
  }
}

```

```

    }
  }
  return route;
}

// -----
// Main route generation
// -----

/**
 * Generate optimized walking routes for a set of properties.
 *
 * Uses the mail-carrier (boustrophedon) algorithm to eliminate backtracking:
 * properties are ordered street-by-street, walking one side then the other,
 * with streets themselves ordered by geographic proximity.
 *
 * @param properties    All candidate properties
 * @param housesPerRoute Target houses per route (currently single-route mode)
 * @param startLocation Rep's starting GPS position
 * @param allLogs       Historical visit logs for cooldown/scoring
 * @param options       Routing options
 * @param learnedWeights ML-derived scoring weights
 * @returns Array of RouteResult objects sorted by competitiveness score
 */
export function generateOptimizedRoutes(
  properties: Property[],
  housesPerRoute = 50,
  startLocation: StartLocation | null = null,
  allLogs: unknown[] = [],
  options: {
    streetCooldownDays?: number;
    useStreetSweep?: boolean;
    minimizeTurns?: boolean;
    use2Opt?: boolean;
    returnToStart?: boolean;
    maxRouteDistance?: number | null;
    excludeTerminal?: boolean;
  } = {},
  learnedWeights: LearnedWeights | null = null,
): RouteResult[] {
  const {
    streetCooldownDays = COOLDOWN_CONFIG.STREET_COOLDOWN_DAYS,
    minimizeTurns = false,
    returnToStart = false,
    maxRouteDistance = null,
    excludeTerminal = true,
  } = options;

```

```

// Filter out properties with invalid coordinates
let eligible = properties.filter(p =>
  p && p.lat && p.lng &&
  !(Math.abs(p.lat) < 0.0001 && Math.abs(p.lng) < 0.0001),
);

// Deduplicate by address key (keep most recent sale)
const addrMap = new Map<string, Property>();
eligible.forEach(p => {
  const street = (p.street_name || "").toUpperCase().trim();
  const num = p.house_number || 0;
  const zip = String(p.zip_code || "").trim().slice(0, 5);
  const key = `${num}|${street}|${zip}`;
  const existing = addrMap.get(key);
  if (!existing) {
    addrMap.set(key, p);
  } else {
    const existDate = existing.sold_date ? new Date(existing.sold_date).getTime() : 0;
    const newDate = p.sold_date ? new Date(p.sold_date).getTime() : 0;
    if (newDate > existDate) addrMap.set(key, p);
  }
});
eligible = Array.from(addrMap.values());

// Apply street cooldown filter
if (allLogs && allLogs.length > 0) {
  const filtered = filterByStreetCooldown(eligible, allLogs, streetCooldownDays);
  eligible = filtered.eligible;
}

// Exclude terminal statuses
if (excludeTerminal) {
  const terminalStatuses = ['HARD_NO', 'DO_NOT_KNOCK', 'COOLDOWN'];
  eligible = eligible.filter(p => !terminalStatuses.includes(p.effective_status ?? ""));
}

if (eligible.length === 0) return [];

// Build neighborhood heat map for scoring
const neighborhoodStats: Record<string, number> = {};
eligible.forEach(p => {
  if (p.lat && p.lng && (
    p.effective_status === 'SOLD' ||
    p.effective_status === 'QUALIFIED' ||
    p.effective_status === 'UNVERIFIED'
  )) {

```

```

try {
  const h3Index = latLngToCell(p.lat, p.lng, 9);
  const disk = gridDisk(h3Index, 1);
  disk.forEach(cell => {
    neighborhoodStats[cell] = (neighborhoodStats[cell] || 0) + 1;
  });
} catch (e) { /* ignore h3 errors */ }
}
});

// Score all properties
const propensityMap = batchScoreProperties(eligible, allLogs, learnedWeights);
const scored: Property[] = eligible.map(p => {
  const hash = p.address_hash || p.id;
  const pData = hash ? propensityMap.get(hash) : undefined;
  const propensity = pData ? pData.propensity : 0.5;
  return {
    ...p,
    propensity,
    score: scoreProperty(p, allLogs as never[], neighborhoodStats, learnedWeights),
  };
});

// Single cluster (single route mode)
const clustered: Property[] = scored.map(p => ({ ...p, cluster: 0 }));
const routes: RouteResult[] = [];
const clusterIds = [...new Set(clustered.map(p => p.cluster))];

for (const i of clusterIds) {
  const clusterProps = clustered.filter(p => p.cluster === i);
  if (clusterProps.length === 0) continue;

  // % % MAIL CARRIER ORDERING (replaces orderForStreetSweep) % %
  let orderedProps = mailCarrierOrder(clusterProps, startLocation);

  // Fatigue-aware front-loading (runs last, respects 12% distance constraint)
  orderedProps = fatigueAwareFrontLoad(orderedProps);

  // Compute route metrics
  let totalDistance = 0;
  let totalScore = 0;
  for (let j = 0; j < orderedProps.length - 1; j++) {
    const legDist = calculateDistance(
      orderedProps[j].lat, orderedProps[j].lng,
      orderedProps[j + 1].lat, orderedProps[j + 1].lng,
    );
    if (maxRouteDistance && (totalDistance + legDist) > maxRouteDistance) {

```

```

        orderedProps.splice(j + 1);
        break;
    }
    totalDistance += legDist;
    totalScore += orderedProps[j].score ?? 0;
}
totalScore += orderedProps[orderedProps.length - 1]?.score ?? 0;

if (returnToStart && orderedProps.length > 1) {
    const returnDist = calculateDistance(
        orderedProps[orderedProps.length - 1].lat, orderedProps[orderedProps.length - 1].lng,
        orderedProps[0].lat, orderedProps[0].lng,
    );
    totalDistance += returnDist;
}

const avgScore = totalScore / orderedProps.length;
const efficiency = orderedProps.length / Math.max(totalDistance, 0.1);

let distanceFromStart = 0;
if (startLocation && orderedProps.length > 0) {
    distanceFromStart = calculateDistance(
        startLocation.lat, startLocation.lng,
        orderedProps[0].lat, orderedProps[0].lng,
    );
}

// Density multiplier
const cellCounts: Record<string, number> = {};
let maxDensity = 0;
orderedProps.forEach(p => {
    if (p.lat && p.lng) {
        try {
            const cell = latLngToCell(p.lat, p.lng, 9);
            cellCounts[cell] = (cellCounts[cell] || 0) + 1;
            if (cellCounts[cell] > maxDensity) maxDensity = cellCounts[cell];
        } catch (e) { /* ignore h3 errors */ }
    }
});

const densityMultiplier = 1 + Math.min(0.2, (maxDensity / 10) * 0.2);
const commutePenalty = Math.min(distanceFromStart * 2, 20);
const competitivenessScore = Math.round(
    ((avgScore * 0.6 + efficiency * 100 * 0.4) * densityMultiplier) - commutePenalty,
);

const routeStreets = [...new Set(orderedProps.map(p => p.street_name).filter(Boolean))] as string[];

```

```

    routes.push({
      id: `route_${i + 1}`,
      name: `Route ${i + 1}`,
      properties: orderedProps,
      houseCount: orderedProps.length,
      streetCount: routeStreets.length,
      streets: routeStreets,
      totalDistance: Math.round(totalDistance * 100) / 100,
      distanceFromStart: Math.round(distanceFromStart * 100) / 100,
      totalScore: Math.round(totalScore),
      avgScore: Math.round(avgScore),
      competitivenessScore,
      status: 'NOT_STARTED',
      completedCount: 0,
    });
  }

  routes.sort((a, b) => b competitivenessScore - a competitivenessScore);
  routes.forEach((route, index) => {
    route.name = `Route ${index + 1}`;
  });

  return routes;
}

// -----
// Re-exports from leadScoring
// -----

export { batchScoreProperties, ownershipDurationScore, SCORING_CONSTANTS } from './leadScoring';

// -----
// Utility exports
// -----

/**
 * Order a flat list of properties by distance (nearest-neighbor + 2-Opt + LinkSwap).
 * Used as a standalone utility — does NOT use the mail-carrier pattern.
 * Kept unchanged for backward compatibility.
 */
export function optimizeRouteByDistance(
  properties: Property[],
  startLocation: StartLocation | null = null,
): Property[] {
  if (!properties || properties.length === 0) return [];
  if (properties.length === 1) return [...properties];

```



```

const props = properties.map(p => ({ ...p }));
const startLat = startLocation?.lat ?? null;
const startLng = startLocation?.lng ?? null;
let ordered = optimizeRouteOrder(props, startLat, startLng, false);
ordered = apply2Opt(ordered);
ordered = applyLinkSwap(ordered);
return ordered;
}

```

/**

* Serialize a route to a JSON-exportable object.

*/

```

export function exportRouteToJSON(route: RouteResult) {
  return {
    route_metadata: {
      route_id: route.id,
      route_name: route.name,
      total_houses: route.houseCount,
      total_distance_miles: route.totalDistance,
      competitiveness_score: route competitivenessScore,
      status: route.status,
      completed: route.completedCount,
      generated_date: new Date().toISOString(),
    },
    properties: route.properties.map((p, idx) => ({
      sequence: idx + 1,
      address_hash: p.address_hash,
      full_address: p.full_address,
      house_number: p.house_number,
      street_name: p.street_name,
      lat: p.lat,
      lng: p.lng,
      status: p.effective_status,
      score: p.score,
      is_ghost: p.is_ghost || false,
    })),
  };
}

```

/**

* Generate an Apple Maps walking directions URL for a route.

* Limited to 10 stops due to Apple Maps URL constraints.

*/

```

export function generateAppleMapsUrl(route: RouteResult): {
  url: string;
  truncated: boolean;
  totalStops: number;
}

```

```

} {
  if (!route.properties || route.properties.length === 0) {
    return { url: "", truncated: false, totalStops: 0 };
  }
  const properties = route.properties;
  const maxStops = Math.min(properties.length, 10);
  const truncated = properties.length > 10;
  const step = Math.max(1, Math.floor(properties.length / maxStops));
  const origin = properties[0];
  const destination = properties[properties.length - 1];
  const waypoints: Property[] = [];
  for (let i = step; i < properties.length - 1; i += step) {
    if (waypoints.length < 8) {
      waypoints.push(properties[i]);
    }
  }
  const originStr = `${origin.lat},${origin.lng}`;
  const destStr = `${destination.lat},${destination.lng}`;
  let url: string;
  if (waypoints.length > 0) {
    const waypointsStr = waypoints.map(p => `${p.lat},${p.lng}`).join('+to:');
    url = `https://maps.apple.com/?saddr=${originStr}&daddr=${waypointsStr}+to:${destStr}&dirflg=w`;
  } else {
    url = `https://maps.apple.com/?saddr=${originStr}&daddr=${destStr}&dirflg=w`;
  }
  return { url, truncated, totalStops: properties.length };
}

```